

Phân Tích Ngữ Nghĩa (Semantics)

Xử Lý Ngôn Ngữ Tự Nhiên
(Natural Language Processing - NLP)
Vũ Đình Khôi

Nội dung

- Khái niệm
 - Ngữ nghĩa từ ngữ (Word meaning)
- Phương pháp biểu diễn ngữ nghĩa
 - Vector semantics & embeddings
- Cơ sở dữ liệu các mối quan hệ từ vựng
- Một số ứng dụng của phân tích ngữ nghĩa

- [1] Chapter 5. Dan Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2026.
 - <https://web.stanford.edu/~jurafsky/slp3/>
 - Embeddings: <https://web.stanford.edu/~jurafsky/slp3/5.pdf>
- [2] Chapter 3. Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, and Harshit Surana, *Practical Natural Language Processing*. O'Reilly, 2020.
- [3] Chapter 2. Jay Alammara and Maarten Grootendorst, *Hands-On Large Language Models*. O'Reilly, 2024.
- [4] Suhas Pai, *Designing Large Language Model Applications*. O'Reilly, 2025.

Chỉ cần đọc các Chapter được liệt kê

I find that I don't understand things unless I try to program them.



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- Richard Feynman

Vector Semantics & Embeddings

Word Meaning

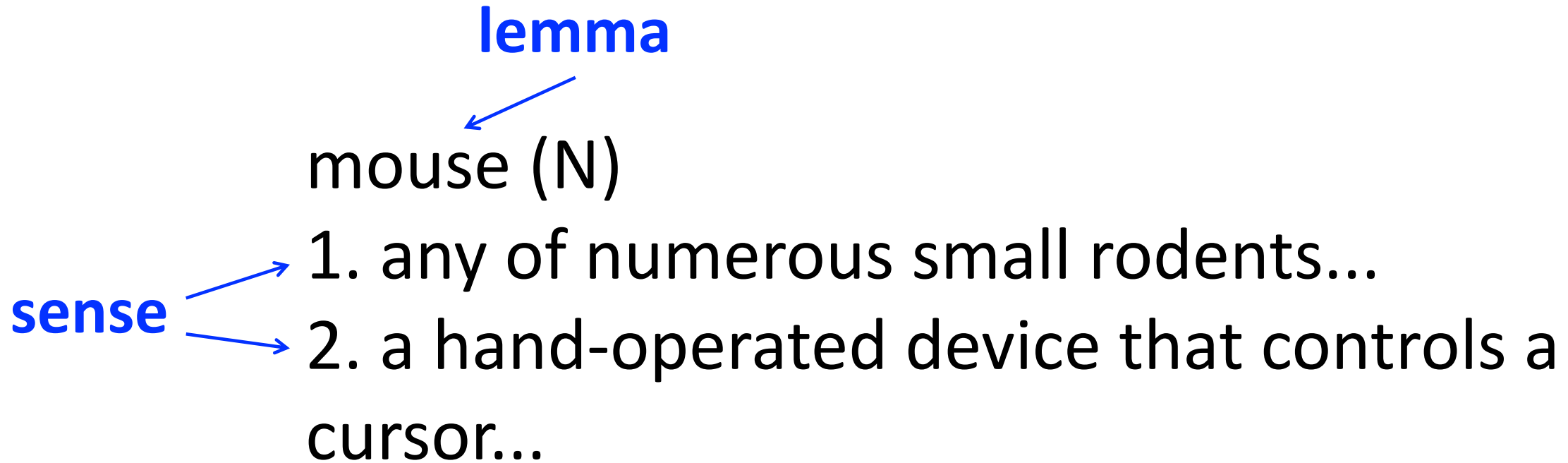
What do words mean?

- N-gram or text classification methods we've seen so far
 - Words are just strings (or indices w_i in a vocabulary list)
 - That's not very satisfactory!
- Introductory logic classes:
 - The meaning of "dog" is DOG; cat is CAT
 - $\forall x \text{ DOG}(x) \rightarrow \text{MAMMAL}(x)$
- Old linguistics joke by Barbara Partee in 1967:
 - Q: What's the meaning of life?
 - A: LIFE
- That seems hardly better!

Desiderata (điều mong muốn thực hiện)

- What should a theory of word meaning do for us?
- Let's look at some desiderata
- From **lexical semantics** (ngữ nghĩa học từ vựng), the linguistic study of word meaning

Lemmas and senses (từ vựng và ý nghĩa)



Modified from the online thesaurus WordNet

A **sense** or “**concept**” is the meaning component of a word
Lemmas can be **polysemous** (have multiple senses)

Relations between senses: Synonymy

- Synonyms (từ đồng nghĩa) have the same meaning in some or all contexts.
 - filbert / hazelnut
 - couch / sofa
 - big / large
 - automobile / car
 - vomit / throw up
 - water / H₂O
- Note that there are probably no examples of perfect synonymy.
 - Even if many aspects of meaning are identical
 - Still may differ based on politeness (phép lịch sự), slang (tiếng lóng), register (cách biểu lộ), genre (thể loại), etc.

Relation: Synonymy?

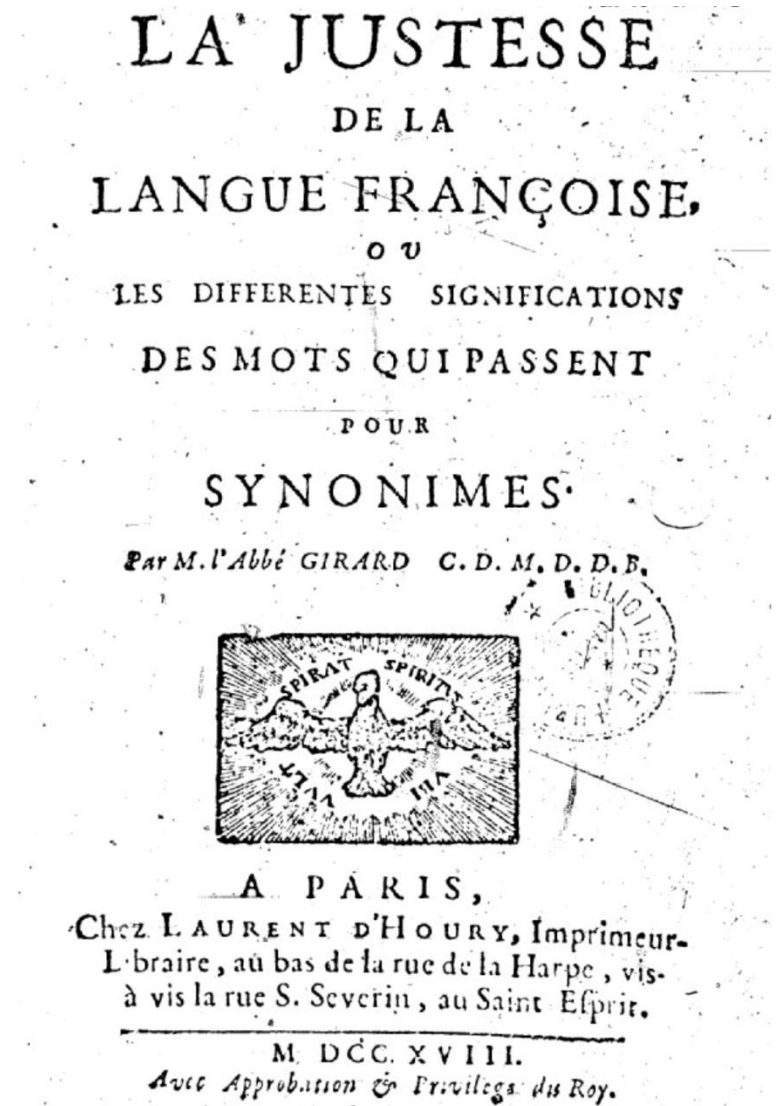
- water/H₂O
 - "H₂O" in a surfing guide?
- big/large
 - my big sister != my large sister
- The Linguistic Principle of Contrast
 - Difference in form → difference in meaning

Abbé Gabriel Girard 1718

Re: "exact" synonyms

"je ne crois pas qu'il y ait de
mot synonyme dans aucune
Langue."

[I do not believe that there is
a synonymous word in any
language]



Relation: **Similarity**

- Words with similar meanings. Not synonyms, but sharing some element of meaning
 - car, bicycle
 - cow, horse

- Ask humans how similar 2 words are

word1	word2	similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al., 2015)

Relation: Word relatedness

- Also called "**word association**"
- Words can be related in any way, perhaps via a semantic frame or field
 - coffee, tea: **similar**
 - coffee, cup: **related**, not similar

Semantic field (trường ngữ nghĩa)

■ Words that

- cover a particular semantic domain
- bear structured relations with each other.

■ hospitals

- *surgeon, scalpel, nurse, anaesthetic, hospital*

■ restaurants

- *waiter, menu, plate, food, menu, chef*

■ houses

- *door, roof, kitchen, family, bed*

Relation: Antonymy (trái nghĩa)

- Senses that are opposites with respect to only one feature of meaning
- Otherwise, they are very similar!
 - dark/light short/long fast/slow rise/fall
 - hot/cold up/down in/out
- More formally: antonyms can
 - define a binary opposition or be at opposite ends of a scale
 - long/short, fast/slow
 - Be *reversives*:
 - rise/fall, up/down

Connotation (sentiment): Nghĩa rộng (cảm xúc)

- Words have **affective** meanings
 - Positive connotations (*happy*)
 - Negative connotations (*sad*)
- Connotations can be subtle:
 - Positive connotation: *copy, replica, reproduction*
 - Negative connotation: *fake, knockoff, forgery*
- Evaluation (sentiment!)
 - Positive evaluation (*great, love*)
 - Negative evaluation (*terrible, hate*)

Connotation

Osgood et al. (1957)

- Words seem to vary along 3 affective dimensions:
 - valence**: the pleasantness of the stimulus
 - arousal**: the intensity of emotion provoked by the stimulus
 - dominance**: the degree of control exerted by the stimulus

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

■ **Concepts** or word senses

- Have a complex many-to-many association with words (homonymy/từ đồng âm, multiple senses)

■ Have relations with each other

- Synonymy (tính đồng nghĩa)
- Antonymy (tính trái nghĩa)
- Similarity (tính tương tự)
- Relatedness (tính họ hàng/liên quan)
- Connotation (nghĩa rộng)

Vector Semantics & Embeddings

Vector Semantics

Computational models of word meaning

- Can we build a theory of how to represent word meaning, that accounts for at least some of the desiderata?
- We'll introduce **vector semantics**
 - The standard model in language processing!
 - Handles many of our goals!

Let's define words by their usages

- One way to define "usage":
 - words are defined by their environments (the words around them)
- Ludwig Wittgenstein
 - PI #43: "The meaning of a word is its use in the language"
- Zellig Harris (1954):
 - **If A and B have almost identical environments we say that they are synonyms.**

What does recent English borrowing *ongchoi* mean?

- Suppose you see these sentences:
 - Ong choi is delicious **sautéed with garlic**.
 - Ong choi is superb **over rice**
 - Ong choi **leaves** with salty sauces
- And you've also seen these:
 - ...spinach **sautéed with garlic over rice**
 - Chard stems and **leaves** are **delicious**
 - Collard greens and other **salty** leafy greens
- Conclusion:
 - Ongchoi is a leafy green like spinach, chard, or collard greens
 - We could conclude this based on words like "leaves" and "delicious" and "sauteed"

Ongchoi: *Ipomoea aquatica* "Water Spinach"

空心菜
kangkong
rau muống
...



Idea 1: Defining meaning by linguistic distribution

- Let's define the meaning of a word by its **distribution** in language use, meaning its neighboring words or grammatical environments.

Idea 2: Meaning as a point in space (Osgood et al. 1957)

- 3 affective dimensions for a word
 - **valence**: pleasantness
 - **arousal**: intensity of emotion
 - **dominance**: the degree of control exerted

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

NRC VAD Lexicon
(Mohammad 2018)

- Hence the connotation of a word is a vector in 3-space

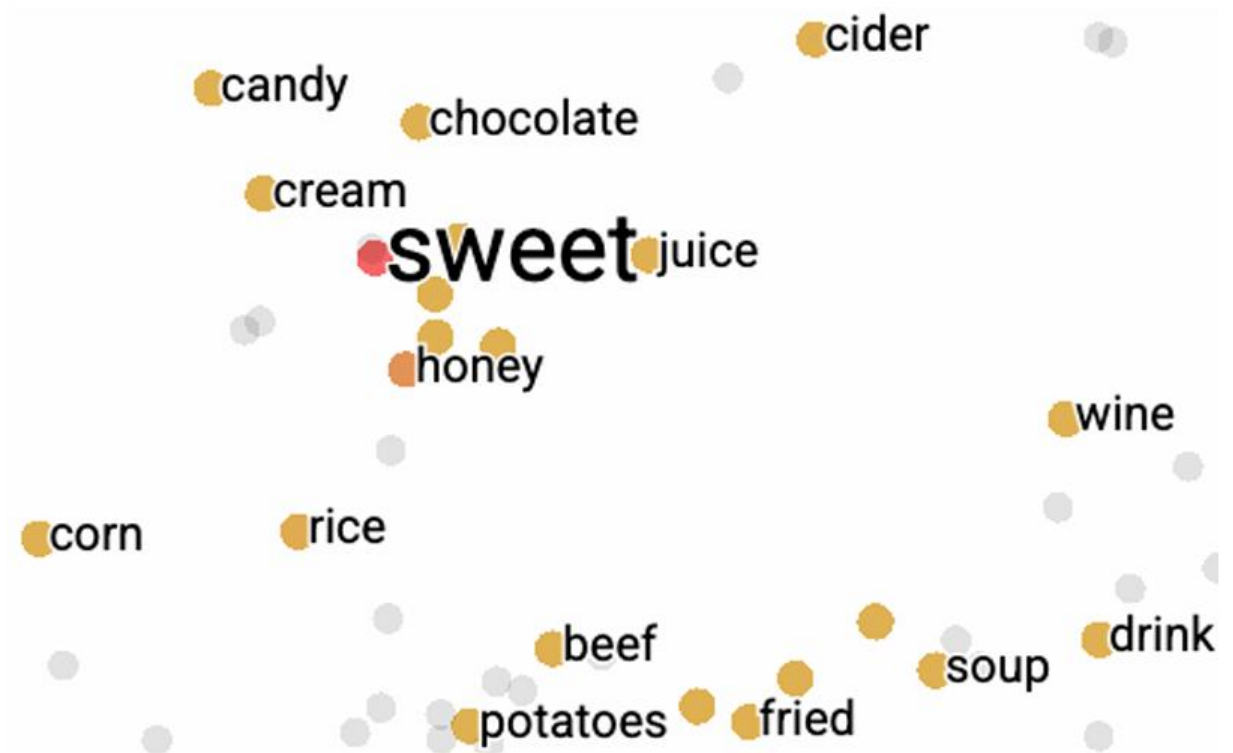
Ideas

- Idea 1: Defining meaning by linguistic **distribution**
- Idea 2: Meaning as a point in **multidimensional space**

Defining meaning as a point in space based on distribution

- Each word = a vector (not just "good" or " w_{45} ")
- Similar words are "nearby in semantic space"
- We build this space automatically by seeing which words are nearby in text

A two-dimensional (t-SNE) visualization of 200-dimensional word2vec embeddings for some words close to the word *sweet*, showing that words with similar meanings are nearby in space. Visualization created using the TensorBoard Embedding Projector <https://projector.tensorflow.org/>.



We define meaning of a word as a vector

- Called an "**embedding**" because it's embedded into a space (see textbook)
- The standard way to represent meaning in NLP
 - Every modern NLP algorithm uses embeddings as the representation of word meaning
- Fine-grained model of meaning for similarity

Intuition: why vectors?

- Consider sentiment analysis:
 - With **words**, a feature is a word identity
 - Feature 5: 'The previous word was "terrible"'
 - requires **exact same word** to be in training and test
 - With **embeddings**:
 - Feature is a word vector
 - 'The previous word was vector [35, 22, 17, ...]'
 - Now in the test set we might see a similar vector [34, 21, 14]
 - We can generalize to **similar but unseen** words!!!

We'll discuss 2 kinds of embeddings

■ Simple count embeddings

- **Sparse** vectors (vector thưa: nhiều giá trị 0)
- Words are represented by the **counts** of nearby words
- Later for information retrieval we'll augment this model to create the **TF-IDF** model of weighting those counts.

■ Word2vec

- **Dense** vectors
- Representation is created by training a classifier to **predict** whether a word is likely to appear nearby
- Later we'll discuss extensions called **contextual embeddings**

From now on:

Computing with meaning representations instead of string representations

荃者所以在鱼，得鱼而忘荃 Nets are for fish;
Once you get the fish, you can forget the net.
言者所以在意，得意而忘言 Words are for meaning;
Once you get the meaning, you can forget the words
庄子(Zhuangzi), Chapter 26

Vector Semantics & Embeddings

Count-based embeddings

Remember the intuition:

- Remember the intuition: words with similar neighborhoods have similar meanings
- How to measure a word's neighborhood?
- Word-context matrix (a kind of co-occurrence matrix)
 - each row represents a word in the vocabulary
 - each column represents how often each other word in the vocabulary appears nearby

Word-Context Matrix

	aardvark	abacus	adept	affect	agate	...	zydeco
aardvark							
abacus							
adept							
affect							
agate							
...							
zydeco							

How often does
agate occur
near **abacus**?

- What does "nearby" mean?
 - For right now let's say "within 4 words"

The word-context matrix

- One set of 4-word contexts

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

The word-context matrix

- Let's consider a mini-matrix of 3 words.
 - How often do "a", "computer", and "pie " occur in the context of "cherry"?

is traditionally followed by	cherry	pie, a traditional dessert
often mixed, such as	strawberry	rhubarb pie. Apple pie
computer peripherals and personal	digital	assistants. These devices usually
a computer. This includes	information	available on the internet

The word-context matrix

- The word-context mini-matrix for just 4 words and 3 contexts
 - Imagine that we are going to look at only the 4 words, and only consider the following 3 context words: *a*, *computer*, and *pie*

is traditionally followed by **cherry** pie, a traditional dessert
 often mixed, such as **strawberry** rhubarb pie. Apple pie
 computer peripherals and personal **digital** assistants. These devices usually
 a computer. This includes **information** available on the internet

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

The word-context matrix

- The word-context mini-matrix for just 4 words and 3 contexts

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

- This 4x3 matrix is a subset of full $|V| \times |V|$ matrix
- Each word is represented by a row vector with dimensionality $[1 \times |V|]$
- With co-occurrence counts with each other word

The word-context matrix

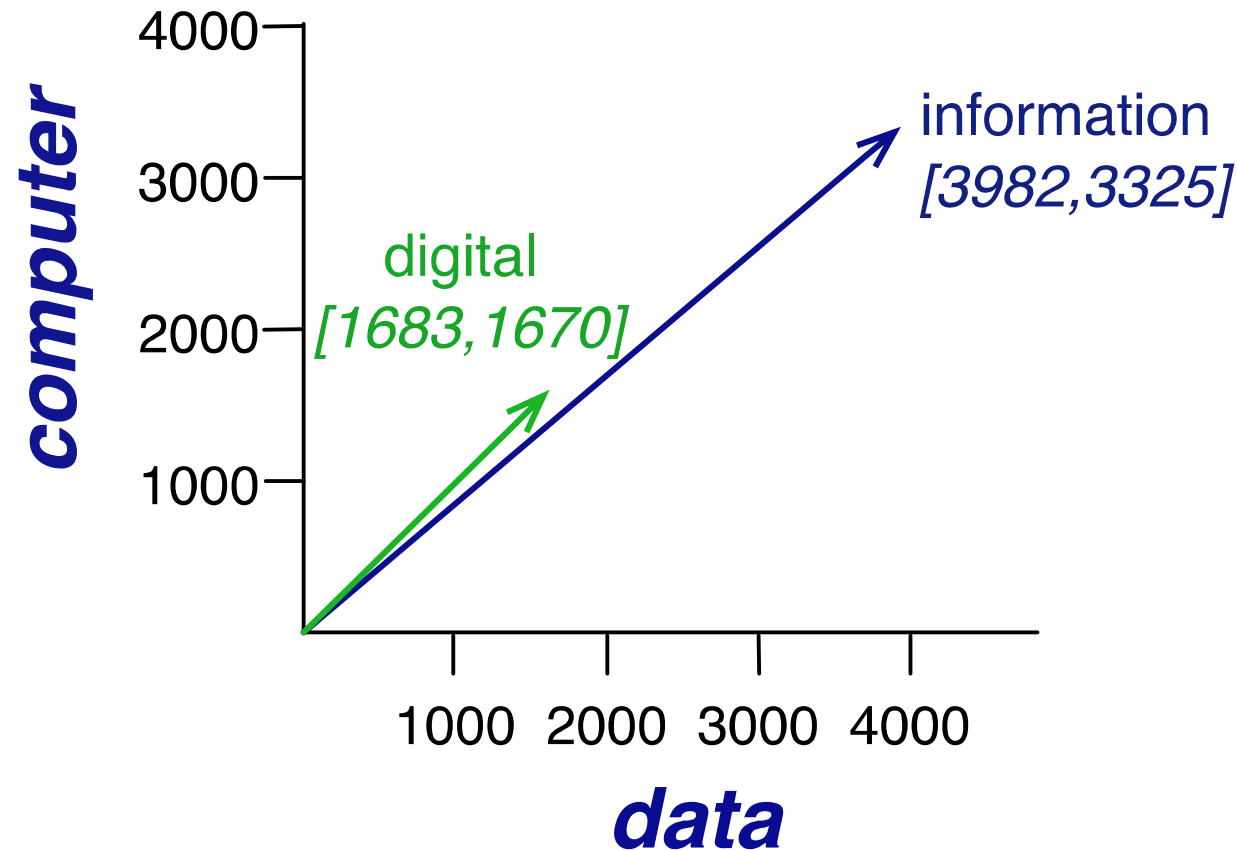
- A selection from a larger word-context matrix

is traditionally followed by **cherry** pie, a traditional dessert
 often mixed, such as **strawberry** rhubarb pie. Apple pie
 computer peripherals and personal **digital** assistants. These devices usually
 a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

The word-context matrix

- A spatial visualization of word vectors for *digital* and *information*, showing just two of the dimensions, corresponding to the words *data* and *computer*



The word-context matrix

- Word context matrix is $|V| \times |V|$
- This could be 50,000 x 50,000
- Most of these numbers are zero!
- So these are **sparse** vectors
- There are efficient algorithms for storing and computing with sparse matrices

Vector Semantics & Embeddings

Cosine for computing word similarity

Computing word similarity: Dot product and cosine

- The dot product (*inner product*) between two vectors is a scalar (tích vô hướng 2 vector):

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

- The dot product tends to be high when the two vectors have large values in the same dimensions
- Dot product can thus be a useful similarity metric between vectors

Problem with raw dot-product

- Dot product favors long vectors
- Dot product is higher if a vector is longer (has higher values in many dimension)
- Vector length (độ lớn/chiều dài vector):

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$

- Frequent words (of, the, you) have long vectors (since they occur many times with other words).
- So dot product overly favors frequent words

Alternative: cosine for computing word similarity

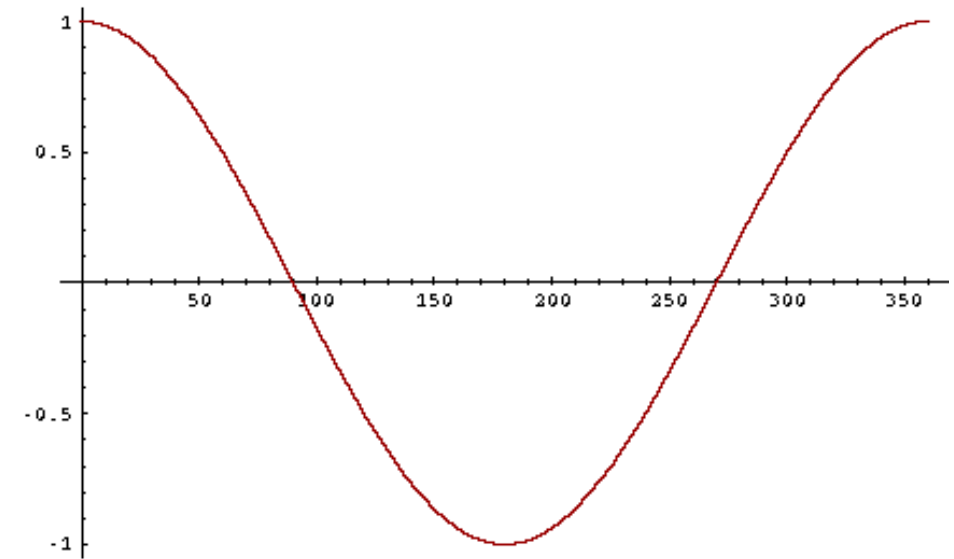
$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

Based on the definition of the dot product between two vectors $\mathbf{a}$ and $\mathbf{b}$

$$\begin{aligned}\mathbf{a} \cdot \mathbf{b} &= |\mathbf{a}| |\mathbf{b}| \cos \theta \\ \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} &= \cos \theta\end{aligned}$$

Cosine as a similarity metric

- -1: vectors point in opposite directions
- +1: vectors point in same directions
- 0: vectors are orthogonal (trực giao)



- But since raw frequency values are non-negative, the cosine for term-term matrix vectors ranges from 0 – 1

Cosine examples

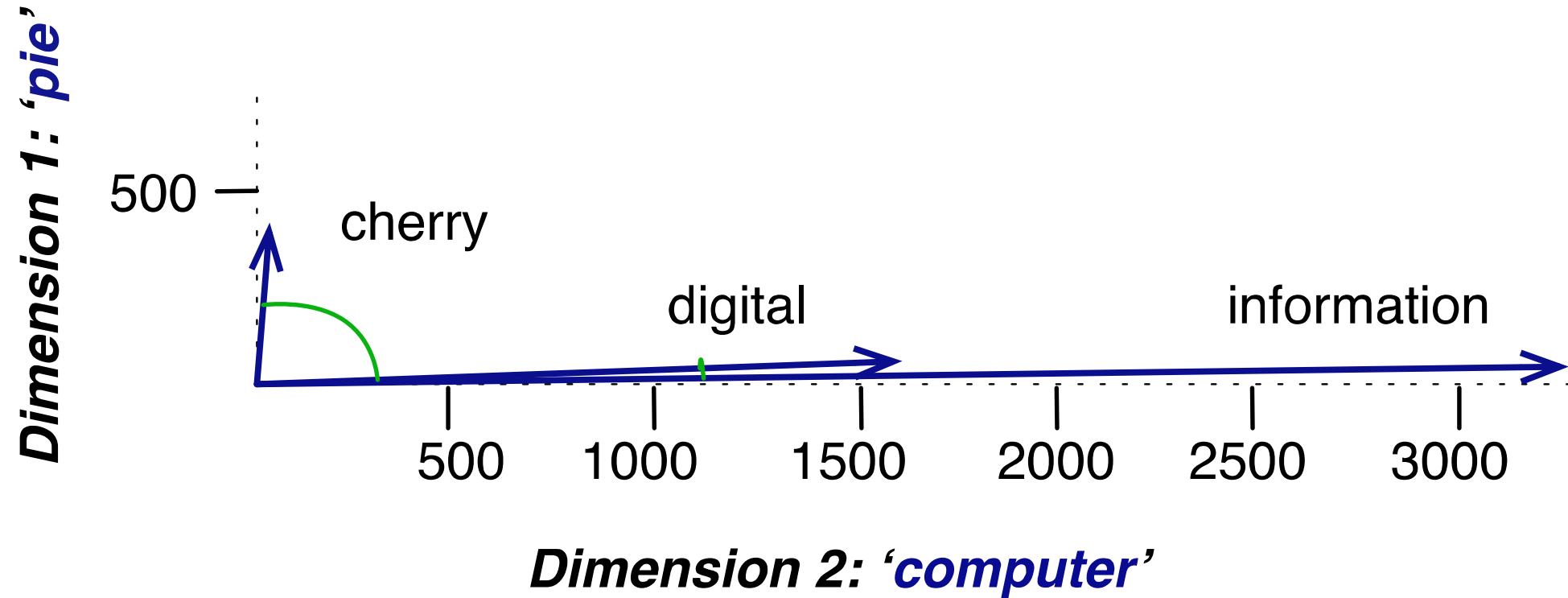
$$\text{cosine}(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\text{cos}(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .018$$

$$\text{cos}(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Visualizing cosines (well, angles)



Vector Semantics & Embeddings

Word2vec

Sparse versus dense vectors

- Count vectors (even if weighted by **TF-IDF**)
 - **long** (length $|V| = 20,000$ to $50,000$)
 - **sparse** (most elements are zero)
- Alternative: learn vectors which are
 - **short** (length $50 - 1000$)
 - **dense** (most elements are non-zero)

Sparse versus dense vectors

■ Why dense vectors?

- Short vectors may be easier to use as **features** in machine learning (fewer weights to tune)
- Dense vectors may **generalize** better than explicit counts
- Dense vectors may do better at capturing synonymy:
 - *car* and *automobile* are synonyms; but are distinct dimensions
 - a word with *car* as a neighbor and a word with *automobile* as a neighbor should be similar, but aren't
- **In practice, they work better**

Common methods for getting short dense vectors

- “Neural Language Model”-inspired models
 - Word2vec (skipgram, CBOW), GloVe
- Singular Value Decomposition (SVD)
 - A special case of this is called LSA – Latent Semantic Analysis
- Alternative to these "static embeddings":
 - Contextual Embeddings (ELMo, BERT)
 - Compute distinct embeddings for a word in its context
 - Separate embeddings for each token of a word

Simple static embeddings you can download!

- Word2vec (Mikolov et al)
 - <https://code.google.com/archive/p/word2vec/>
- GloVe (Pennington, Socher, Manning)
 - <http://nlp.stanford.edu/projects/glove/>

Word2vec

- Popular embedding method
 - Very fast to train
 - Code available on the web
- Idea: **predict** rather than **count**
- Word2vec provides various options. We'll do:
 - skip-gram with negative sampling (SGNS)

Word2vec

- Instead of **counting** how often each word w occurs near "*apricot*"
 - Train a classifier on a binary **prediction** task:
 - Is w likely to show up near "*apricot*"?
- We don't actually care about this task
 - But we'll take the learned classifier weights as the word embeddings
- Big idea: **self-supervision**
 - A word c that occurs near *apricot* in the corpus acts as the gold "correct answer" for supervised learning
 - No need for human labels
 - Bengio et al. (2003); Collobert et al. (2011)

Approach: predict if candidate word c is a "neighbor"

1. Treat the target word t and a neighboring context word c as **positive examples**.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

Skip-Gram Training Data

- Assume a ± 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [word w] c3 c4

Skip-Gram Classifier

- (assuming a +/- 2 word window)

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [w] c3 c4

- Goal: train a classifier that is given a candidate (**w**ord, **c**ontext) pair

(apricot, jam)

(apricot, aardvark)

...

- And assigns each pair a probability:

$$P(+|w, c)$$

$$P(-|w, c) = 1 - P(+|w, c)$$

Similarity is computed from dot product

- Remember: two vectors are similar if they have a high dot product
 - Cosine is just a normalized dot product
- So:
 - $\text{similarity}(w, c) \propto w \cdot c$
- We'll need to normalize to get a probability
 - (cosine isn't a probability either)

Turning dot products into probabilities

- $\text{sim}(w, c) \approx w \cdot c$
- To turn this into a probability
- We'll use the sigmoid from logistic regression: $\sigma(x) = \frac{1}{1 + \exp(-x)}$

$$P(+|w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

$$\begin{aligned} P(-|w, c) &= 1 - P(+|w, c) \\ &= \sigma(-\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(\mathbf{c} \cdot \mathbf{w})} \end{aligned}$$

How Skip-Gram Classifier computes $P(+|w, c)$

$$P(+|w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

- This is for one context word, but we have lots of context words.
- We'll assume independence and just multiply them:

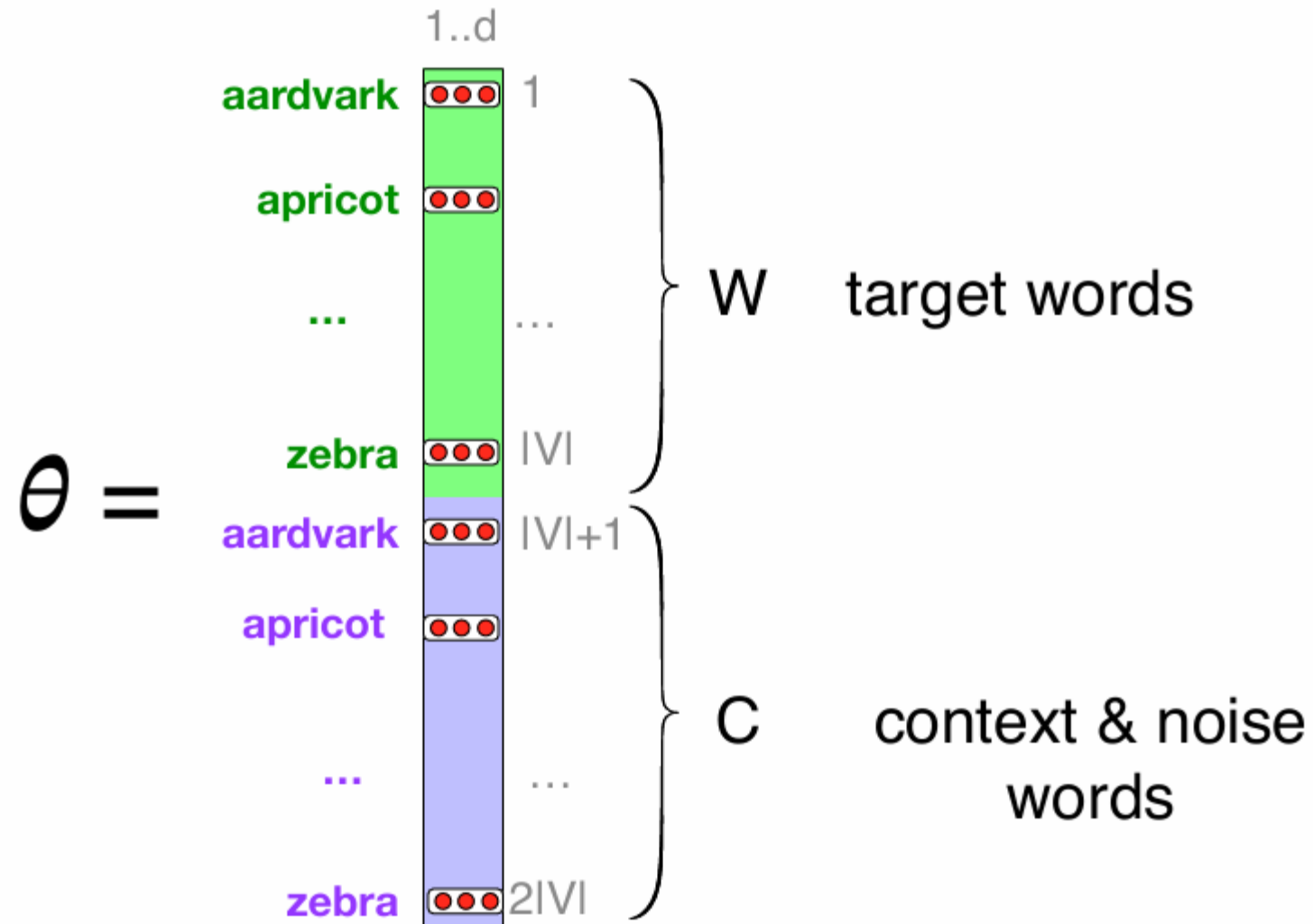
$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

Skip-gram classifier: summary

- A probabilistic classifier, given
 - a test target word w
 - its context window of L words $c_{1:L}$
- Estimates probability that w occurs in this window based on similarity of w (embeddings) to $c_{1:L}$ (embeddings).
- To compute this, we just need embeddings for all the words.

These embeddings we'll need: a set for w , a set for c



Vector Semantics & Embeddings

Word2vec: Learning the embeddings

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1

c2

[w]

c3

c4



positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1

c2

[w]

c3

c4



positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

For each positive example we'll grab k negative examples, sampling by frequency

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1

c2

[w]

c3

c4



positive examples +

w	c _{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c _{neg}	w	c _{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Word2vec: how to learn vectors

- Given the set of positive and negative training instances, and an initial set of embedding vectors
- The goal of learning is to adjust those word vectors such that we:
 - **Maximize** the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive data
 - **Minimize** the similarity of the (w, c_{neg}) pairs drawn from the negative data.

Loss function for one w with c_{pos} , $c_{neg1} \dots c_{negk}$

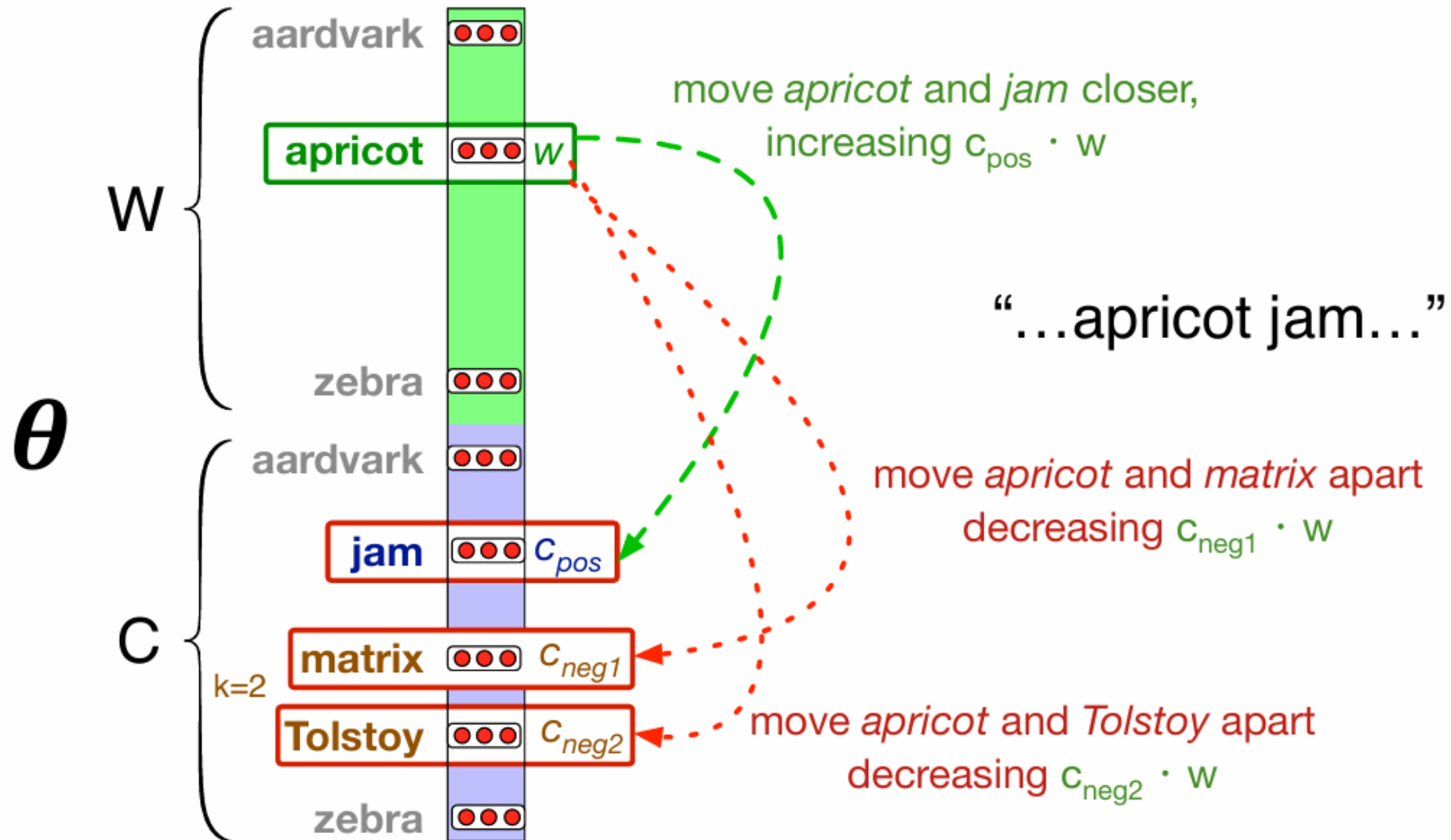
- Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the k negative sampled non-neighbor words.

$$\begin{aligned}
 L(w, c_{pos}, c_{neg*}) &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\
 &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\
 &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\
 &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]
 \end{aligned}$$

Learning the classifier

- How to learn?
 - Stochastic gradient descent!
- We'll adjust the word weights to
 - make the positive pairs more likely
 - and the negative pairs less likely,
 - over the entire training set.

Intuition of one step of gradient descent



Reminder: gradient descent

■ At each step

- Direction: We move in the reverse direction from the gradient of the loss function
- Magnitude: we move the value of this gradient $\frac{d}{dw} L(f(x; w), y)$ weighted by a **learning rate** η
- Higher learning rate means move w faster

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

The derivatives of the loss function

$$L(w, c_{pos}, c_{neg}^*) = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

$$\frac{\partial L}{\partial c_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{w}$$

$$\frac{\partial L}{\partial c_{neg_i}} = [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{w}$$

$$\frac{\partial L}{\partial w} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{c}_{neg_i}$$

Update equation in SGD

- Start with randomly initialized $\mathbf{C}$ and $\mathbf{W}$ matrices, then incrementally do updates

$$\mathbf{c}_{pos}^{t+1} = \mathbf{c}_{pos}^t - \eta [\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{w}^t$$

$$\mathbf{c}_{neg_i}^{t+1} = \mathbf{c}_{neg_i}^t - \eta [\sigma(\mathbf{c}_{neg_i}^t \cdot \mathbf{w}^t)] \mathbf{w}^t$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \left[[\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{c}_{pos}^t + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i}^t \cdot \mathbf{w}^t)] \mathbf{c}_{neg_i}^t \right]$$

Two sets of embeddings

- SGNS learns two sets of embeddings
 - Target embeddings matrix W
 - Context embedding matrix C
- It's common to just add them together, representing word i as the vector $W_i + C_i$

Summary: How to learn word2vec (skip-gram) embeddings

- Start with V random d -dimensional vectors as initial embeddings
- Train a classifier based on embedding similarity
 - Take a corpus and take pairs of words that co-occur as positive examples
 - Take pairs of words that don't co-occur as negative examples
 - Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
 - Throw away the classifier code and keep the embeddings.

Vector Semantics & Embeddings

Properties of Embeddings

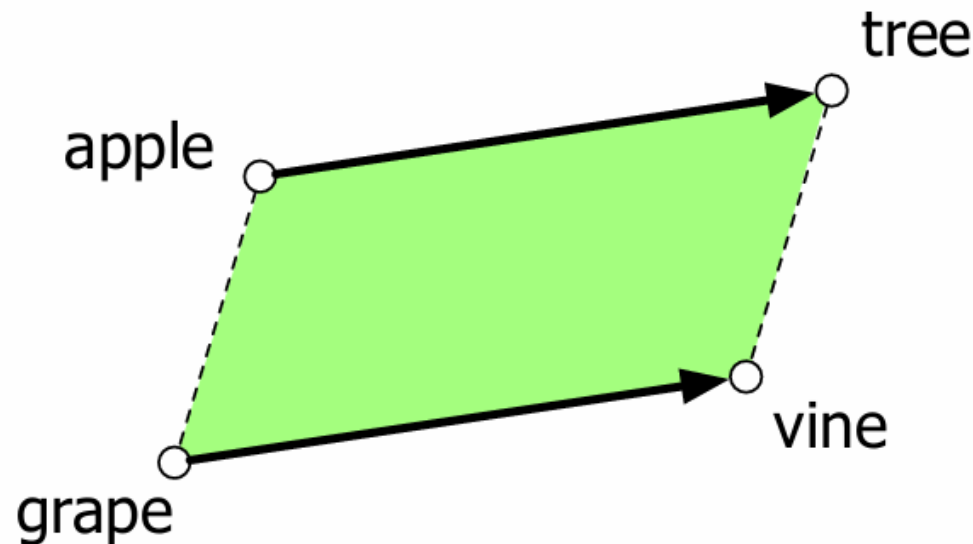
The kinds of neighbors depend on window size

- **Small windows** ($C = \pm 2$): nearest words are syntactically similar words in same taxonomy (cùng nguyên tắc phân loại)
 - *Hogwarts* nearest neighbors are other fictional schools:
 - *Sunnydale, Evernight, Blandings*
- **Large windows** ($C = \pm 5$): nearest words are related words in same semantic field
 - *Hogwarts* nearest neighbors are Harry Potter world:
 - *Dumbledore, half-blood, Malfoy*

Analogical relations

- The classic parallelogram model of analogical reasoning (Rumelhart and Abrahamson 1973)
- To solve: "*apple is to tree as grape is to _____*"

- Add *tree* – *apple* to *grape* to get *vine*



- Cây táo (*apple*) là cây gỗ (*tree*) cũng như cây nho (*grape*) là cây thân leo (*vine*)
- Tất cả tính toán trên các vector

Analogical relations via parallelogram

- The parallelogram method can solve analogies with both sparse and dense embeddings (Turney and Littman 2005, Mikolov et al. 2013b)

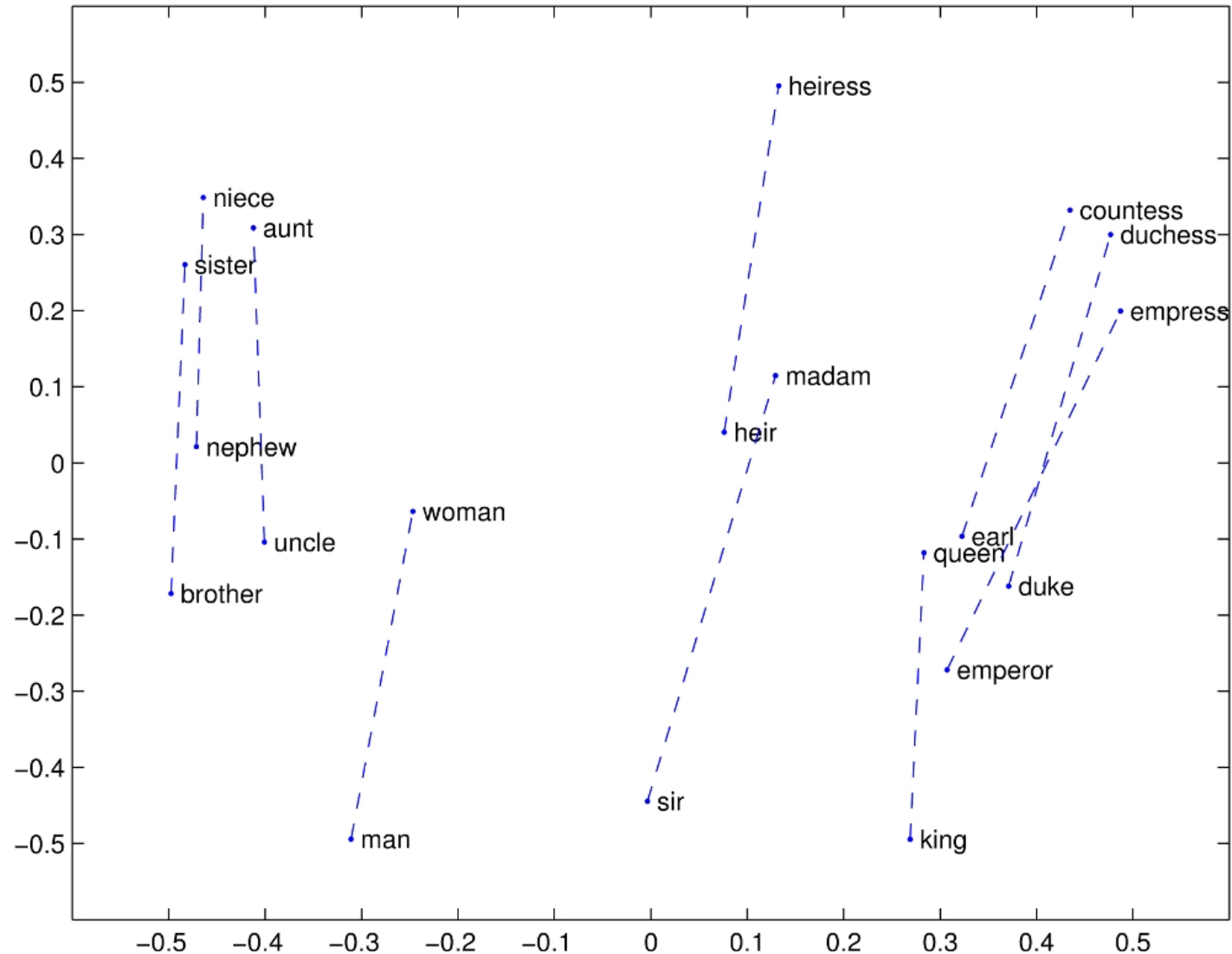
$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$ is close to $\overrightarrow{\text{queen}}$

$\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}}$ is close to $\overrightarrow{\text{Rome}}$

- For a problem $a : b :: a^* : b^*$, the parallelogram method is:

$$\hat{\mathbf{b}}^* = \underset{\mathbf{x}}{\operatorname{argmin}} \operatorname{distance}(\mathbf{x}, \mathbf{b} - \mathbf{a} + \mathbf{a}^*)$$

Structure in GloVe Embedding space



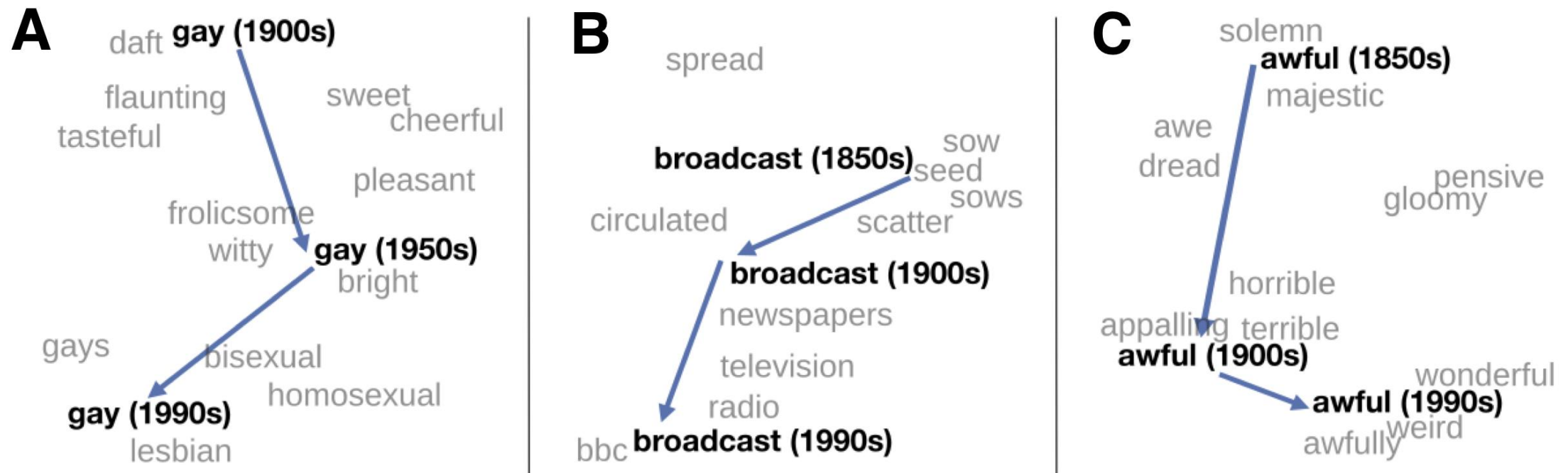
Caveats with the parallelogram method

- It only seems to work for frequent words, small distances and certain relations (relating countries to capitals, or parts of speech), but not others. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)
- Understanding analogy is an open area of research (Peterson et al. 2020)

Embeddings as a window onto historical semantics

- Train embeddings on different decades of historical text to see meanings shift

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Embeddings reflect cultural bias!

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

- Ask “Paris : France :: Tokyo : x”
 - x = Japan
- Ask “father : doctor :: mother : x”
 - x = nurse
- Ask “man : computer programmer :: woman : x”
 - x = homemaker

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. Proceedings of the National Academy of Sciences 115(16), E3635–E3644.

- Compute a **gender** or **ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities
 - Embeddings for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical*) are biased toward men, a bias slowly decreasing 1960-1990
 - Embeddings for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20th century.
- These match the results of old surveys done in the 1930s

